DECAF20 Scanner Submission 1

Christopher Harris
CS 4308 Concepts of Programming Languages – Section 01
College of Computing and Software Engineering
Kennesaw State University
Marietta, Georgia
charr396@students.kennesaw.edu
October 5, 2026

I. Problem

The DECAF20 programming language has a specific grammar it must adhere to. Before a parser can verify that the code is syntactically and logically correct according to this grammar, the tokens that make up the program must first be identified and saved. That is the purpose of the scanner. It iterates through the given DECAF20 file to collect each token, including string constants, integer constants, identifiers, keywords, operators, and other symbols. This is the first step of the infrastructure needed before verifying the code, because it must be gathered and saved in an organized manner first.

II. Purpose and Summary

The purpose of this deliverable was to develop a Python scanner for the DECAF20 programming language. To build it, a straightforward implementation was introduced, with the logic iterating through the given file line-by-line and, in each line, character-by-character in order to collect tokens.

A Token class was created to save key attributes of each token, including its value, token type, line number, and start and end columns. The class also contains a method to format and neatly print them out. Additionally, multiple dictionary lists were defined for keyword identifiers, two-character operators, and additional single-character characters that are part of the DECAF20 language.

The Scanner class does the actual scanning. First, it takes in the filename to scan and creates an empty list for storing each token. Then, it opens the file, enumerating through it line-by-line. In each line scan, a column number tracker is utilized to step through character-by-character until the end of the line is reached. The code is equipped to handle each reached token indication. Whitespace is skipped, and dedicated methods are included to scan strings, integers, and identifiers based on the respective indications of quotation marks, digits, and letters. The scanner also checks for two-character operators before checking for single-character tokens, covering the entire DECAF20 language as a result. Any comments indicated by double forward slashes are skipped over as well.

Each scanned token is stored in order of appearance in a list. After scanning, the Scanner class also has a print method that simply calls the print function of each token in the accumulated list for display.

III. Test Input and Output Results

The following screenshot demonstrates the executed scanner output of one test file, whose DECAF20 source code is also shown below. In the output, all tokens were identified by the scanner along with all the key attribute data.

```
void main() {
  int c;
  string s;

  s = "hello";
  c = test(4, 5);
  Print(c);
  Print(s);
}

int test(int a, int b) {
  return a + b;
}
```

```
PS C:\Users\chris\PycharmProjects\DECAF20-scanner\src> python scanner.py tests/t1.decaf
void     line 2 Cols  1 - 4  is  T_Void
main     line 2 Cols  6 - 9  is  T_IDENTIFIER
(     line 2 Cols  10 - 10  is  '('
)     line 2 Cols  11 - 11  is  ')'
{     line 2 Cols  13 - 13  is  '{'
int     line 3 Cols  3 - 5  is  T_Int
c     line 3 Cols  7 - 7  is  T_IDENTIFIER
;     line 3 Cols  8 - 8  is  ';'
string     line 4 Cols  3 - 8  is  T_String
s     line 4 Cols  10 - 10  is  T_IDENTIFIER
;     line 4 Cols  11 - 11  is  ';'
s     line 7 Cols  3 - 3  is  T_IDENTIFIER
=     line 7 Cols  5 - 5  is  '='
"hello"     line 7 Cols  7 - 13  is  T_StringConstant (value= "hello")
;     line 7 Cols  14 - 14  is  ';'
c     line 8 Cols  3 - 3  is  T_IDENTIFIER
=     line 8 Cols  5 - 5  is  '='
test     line 8 Cols  7 - 10  is  T_IDENTIFIER
(     line 8 Cols  11 - 11  is  '('
4     line 8 Cols  12 - 12  is  T_IntConstant (value= 4)
,     line 8 Cols  13 - 13  is  ','
5     line 8 Cols  15 - 15  is  T_IntConstant (value= 5)
)     line 8 Cols  16 - 16  is  ')'
;     line 8 Cols  17 - 17  is  ';'
Print     line 9 Cols  3 - 7  is  T_Print
(     line 9 Cols  8 - 8  is  '('
c     line 9 Cols  9 - 9  is  T_IDENTIFIER
)     line 9 Cols  10 - 10  is  ')'
;     line 9 Cols  11 - 11  is  ';'
Print     line 10 Cols  3 - 7  is  T_Print
(     line 10 Cols  8 - 8  is  '('
s     line 10 Cols  9 - 9  is  T_IDENTIFIER
)     line 10 Cols  10 - 10  is  ')'
;     line 10 Cols  11 - 11  is  ';'
}     line 11 Cols  1 - 1  is  '}'
int     line 13 Cols  1 - 3  is  T_Int
test     line 13 Cols  5 - 8  is  T_IDENTIFIER
(     line 13 Cols  9 - 9  is  '('
int     line 13 Cols  10 - 12  is  T_Int
a     line 13 Cols  14 - 14  is  T_IDENTIFIER
,     line 13 Cols  15 - 15  is  ','
int     line 13 Cols  17 - 19  is  T_Int
b     line 13 Cols  21 - 21  is  T_IDENTIFIER
)     line 13 Cols  22 - 22  is  ')'
{     line 13 Cols  24 - 24  is  '{'
return     line 14 Cols  3 - 8  is  T_Return
a     line 14 Cols  10 - 10  is  T_IDENTIFIER
+     line 14 Cols  12 - 12  is  '+'
b     line 14 Cols  14 - 14  is  T_IDENTIFIER
;     line 14 Cols  15 - 15  is  ';'
}     line 15 Cols  1 - 1  is  '}'
```

Fig. 1. Example Program Output

IV. Conclusion

The completed scanner accomplishes the first component of the pipeline needed to process the DECAF20 language. By storing each token in this organized structure, the scanner provides the parser with all the critical information pertaining to each token, including where it appears in the code, its token type, and its value. This will be vital for the next stage of the project.

Implementing the scanner provided the foundation for understanding how source code is transformed into tokens for parsing. This is vital for the next stage of the project, which will involve building upon the scanner by enforcing error catching by the parser.

References

[1] M. Alghoul, "Building a Custom Parser: A Step-by-Step Guide," *Medium*, Aug. 5, 2023. [Online]. Available: https://medium.com/@menta8seca/building-a-custom-parser-a-step-by-step-guide-b0a92340ff51

[2] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Pearson, 2019.